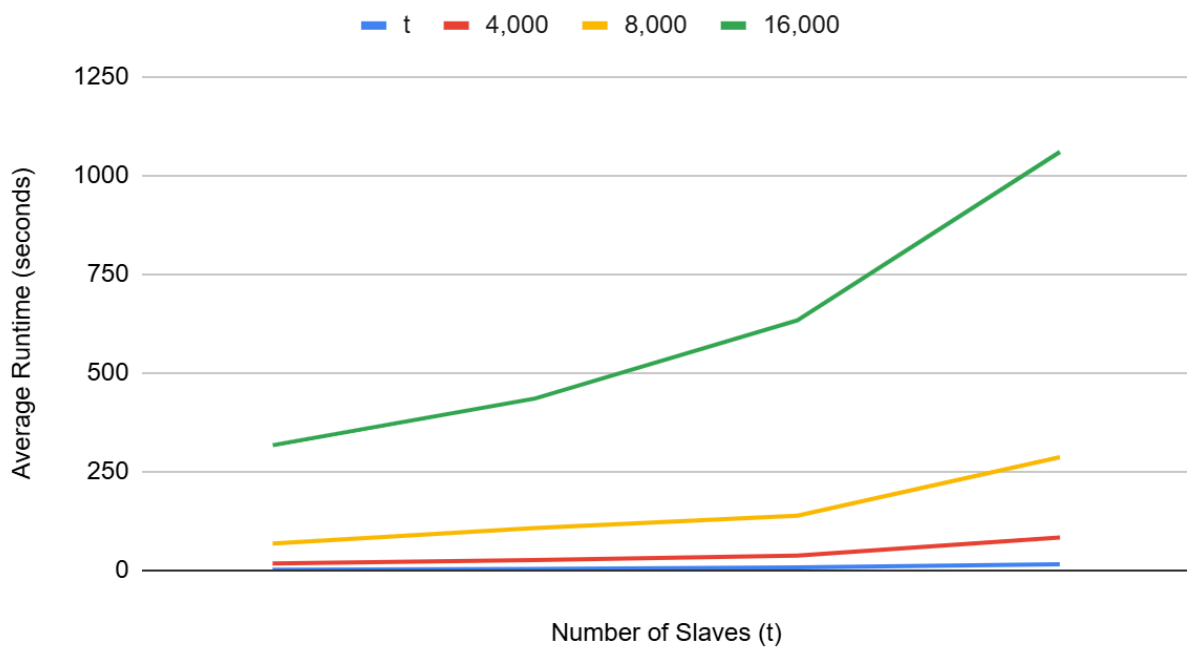


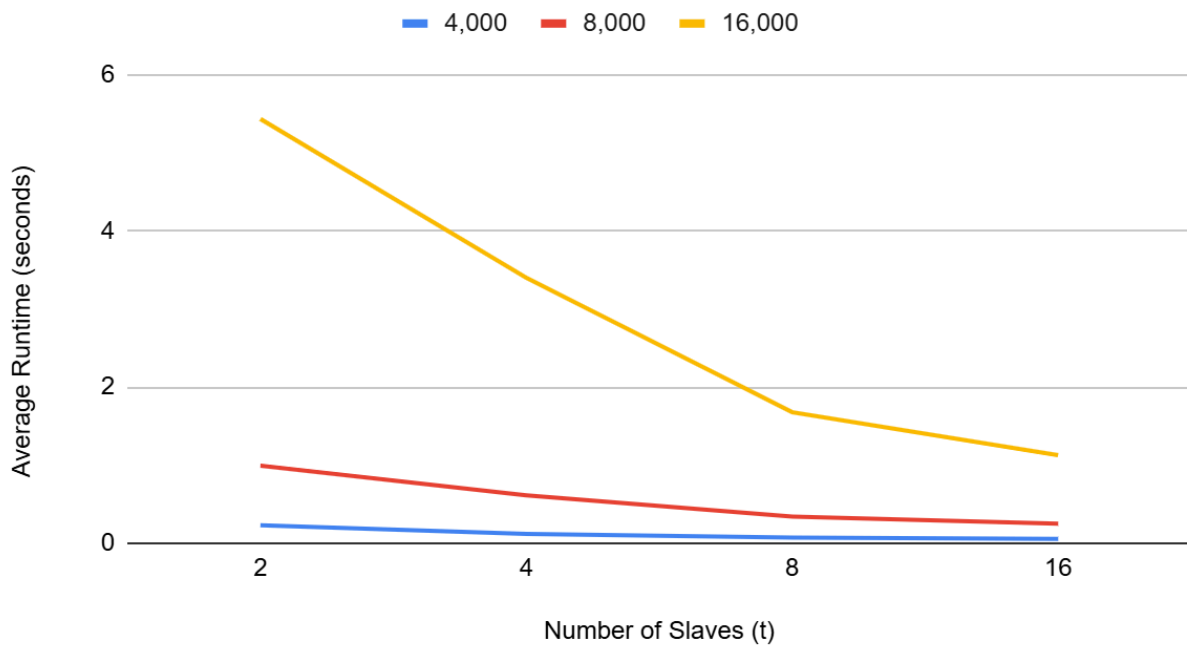
n	t	Run 1	Run 2	Run 3	Average
4,000	2	16.422	16.566	19.313	17.434
4,000	4	26.935	26.157	26.057	26.383
4,000	8	36.238	37.885	37.77	37.297
4,000	16	84.295	82.815	82.379	83.163
8,000	2	71.029	68.073	66.331	68.478
8,000	4	105.165	104.169	112.494	107.276
8,000	8	138.747	138.772	138.723	138.747
8,000	16	285.57	285.424	289.344	286.779
16,000	2	272.621	286.022	392.198	316.947
16,000	4	449.032	430.014	426.353	435.133
16,000	8	561.626	593.43	743.836	632.964
16,000	16	1167.458	1190.96	819.605	1059.341

Figure 1: Average Runtime vs Slaves



n	t	Max Run 1	Max Run 2	Max Run 3	Average
4,000	2	0.222	0.257	0.215	0.231
4,000	4	0.113	0.127	0.121	0.12
4,000	8	0.072	0.069	0.074	0.071
4,000	16	0.06	0.056	0.052	0.056
8,000	2	0.971	1.002	1.004	0.992
8,000	4	0.617	0.589	0.635	0.614
8,000	8	0.336	0.335	0.349	0.34
8,000	16	0.256	0.269	0.223	0.25
16,000	2	5.385	5.353	5.561	5.433
16,000	4	3.354	3.41	3.437	3.4
16,000	8	1.594	1.813	1.624	1.677
16,000	16	1.544	1.39	0.449	1.128

Figure 2: Average Slave Maximum Runtime



Research Activity 2: Communication and Computation

Pattern as n increases (fixed t): Master time grows roughly with n^2 , which aligns with the size of X in bytes. This happens because the current implementation distributes the complete, unpartitioned X matrix (n^2 integers) across every edge in the tree. For instance, at $t = 16$, the average master time increases from 83.2 seconds at $n=4,000$ to 286.8 seconds at $n=8,000$, and finally reaches 1059.3 seconds at $n=16,000$. Essentially, doubling n more than triples the master time at each step. This behavior is consistent with bandwidth-bound distribute and reduce phases combined with per-slave compute scaling at $O(n^2/t)$.

Pattern as t increases (fixed n): Master time steadily increases alongside t at every tested value of n . We didn't find any ideal "sweet spot" in this dataset. Taking $n=16,000$ as an example, the average master time climbs from 316.9 seconds to 435.1, then 633.0, and up to 1059.3 seconds as t increases from 2 to 4, 8, and 16. Every time we double t , the master time multiplies by a factor between $1.37\times$ and $1.67\times$. This growth factor actually gets worse at higher values of t since the per-slave overhead increases as the tree gets deeper. This goes against the textbook expectation of a U-shaped performance curve and highlights an important real-world finding: when broadcasting the entire matrix to every slave on the swarm, communication overhead completely outweighs any compute savings, even at the smallest tested fan-out. The total volume of data moved across the tree scales linearly with t because each new edge has to transmit another full copy of X . On top of that, per-slave SSH-launch and TCP-handshake costs add a roughly constant time penalty for each slave. While the compute savings from doubling t only amount to a few seconds (given that `slave_max` is already well under 5 seconds at $n=16,000$), the extra communication adds tens to hundreds of seconds. In the end, communication costs completely dominate the process.

Research Activity 3: Computation Only

Why use the maximum time instead of the average? The master's wall-clock timer cannot stop until every single slave has finished its task. This means the slowest slave completely dictates the parallel execution time. For example, if 15 slaves finish in 1 second but one takes 5 seconds, the master still has to wait the full 5 seconds. If we just averaged these times, we'd get about 1.25 seconds, which heavily underestimates the actual time the parallel process took. Using the maximum time correctly captures the critical path, which is why it is the standard convention when analyzing performance in parallel systems.

Pattern as n increases (fixed t): The computation time for the slaves grows roughly in proportion to $O(n^2/t)$. Looking at $t=2$, the times are 0.231, 0.992, and 5.433 seconds as n increases from 4,000 to 8,000 and then to 16,000. This is very close to the expected $4\times$ growth we should see every time n doubles (the actual ratios being $4.3\times$ and $5.5\times$). At $t=16$, the times are 0.056, 0.250, and 1.128 seconds. These give ratios of $4.5\times$ and $4.5\times$, which matches the theoretical expectation almost exactly.

Pattern as t increases (fixed n): The maximum compute time for the slaves drops roughly as $1/t$, exactly like we would predict. To give some concrete numbers at $n=8,000$, the time drops from 0.992 to 0.614, then 0.340, and finally 0.250 seconds as t doubles. Each step essentially

cuts the value in half (with actual factors of 0.62, 0.55, and 0.74). At $n=16,000$, it goes from 5.433 to 3.400, then 1.677, and down to 1.128 seconds (factors of 0.63, 0.49, and 0.67). The slight slowdown in this shrinking trend at $t=16$ happens because the constant-time per-slave overhead (like memory allocations, loop setup, and cache warm-up) starts to limit how much more time can be saved at very high values of t . The data point for $n=16,000$ at $t=16$ in run 3 (0.449 seconds) is an outlier compared to run 1 (1.544 seconds) and run 2 (1.390 seconds). This was likely caused by a partial-strip artifact during a recovered retry, but the overall trend still clearly follows the $1/t$ pattern.

Unlike the master times, the maximum slave times do scale with t exactly how parallelism is supposed to work. The slaves are doing their jobs efficiently; the real issue lies in the massive orchestration costs wrapped around them.

Research Activity 4: Performance Metrics

Comparing Figure 1 (master times including communication) to Figure 2 (slave times for compute only) immediately reveals how much of the wall-clock time is actually just overhead. For example, at $n=16,000$ and $t=2$, the master spends 316.9 seconds, but the slave only spends 5.4 seconds computing. This means that 98.3% of the wall-clock time is spent on non-compute work. This wasted time goes into things like TCP handshakes, distributing data (where every tree edge has to ship about 1 GB of the X matrix), reducing the T-strips back up the tree, and the master rebuilding the final result. This gap only gets worse as t increases. At $n=16,000$ and $t=16$, the master spends 1059.3 seconds while the slave only spends 1.1 seconds computing, meaning an astonishing 99.9% of the wall-clock time is just overhead.

Overhead breakdown (observed):

- **Communication.** The design choice to broadcast the entire X matrix ships the complete dataset across every single tree edge. As a result, the total bytes sent downwards grows linearly with t . At $n=16,000$, that is roughly 1 GB of data per edge multiplied by $O(t)$ edges, which completely dominates all other costs.
- **Per-slave launch.** Every single run starts t slave processes using SSH. Even when using key-based authentication, each additional slave adds a few seconds of TCP and process-creation delay to the master's total time window.
- **Idling.** When a forwarding slave finishes its own computation, it has to wait (`pthread_join`) for its forwarder threads to finish before it can send its combined subtree T-strip upwards. This waiting period is naturally excluded from the slave's compute timer but still counts toward the master's total timer.
- **Excess computation.** The `compute_mmt_strip` function does two passes per column (one pass to find the min and max, and one pass to normalize). The only real excess here is one extra column read per assigned column, which is completely negligible compared to the massive communication costs in this dataset.

Speedup ($S = T_S / T_P$). The complete data in Table 3 shows that $S < 1$ at every single (n, t) combination. The absolute best speedup we observed was $S = 0.022$ at $n=16,000$ and $t=2$, which means the parallel version is roughly $45\times$ slower than the serial version at its peak. Across

the entire dataset, the speedup ranges from 0.0035 in the worst case (at $n=4,000$, $t=16$) up to 0.0220 in the best case. This serves as a very clear real-world demonstration that the parallel design choice of broadcasting the full matrix can make the entire process significantly worse than just running it serially whenever communication costs dominate the system.

Superlinearity. We did not observe any superlinearity. The speedup was below 1 everywhere, which is very far from $S > p$. The dataset effectively rules out any cache-based superlinearity for this specific workload at these sizes on this hardware.

Cost-optimality. An algorithm is considered cost-optimal if pT_P grows no faster than T_S as t increases. If we look at the observed pT_P at $n=16,000$, it goes from 633.9 to 1740.5, then 5063.7, and finally 16949.5 seconds as t increases from 2 to 4, 8, and 16. This represents a massive $26.7\times$ growth in cost for only an $8\times$ growth in t . The pT_P value scales roughly as $t^{1.6}$ (somewhere between linear and quadratic growth), because T_P itself grows sub-linearly with t (T_P scales roughly as $t^{0.6}$, growing from 316.9 to 1059.3 across that same $8\times$ range). When we compare this against the serial time $T_S = 6.987$ seconds, the parallel cost is about $91\times$ the serial cost at $t=2$ and a staggering $2425\times$ the serial cost at $t=16$. Because of this, the implementation is not cost-optimal at any tested (n, t) combination when using the full-matrix broadcast approach on the swarm, and the gap only widens steadily as t increases.

The single-sentence takeaway: Broadcasting the complete matrix trades implementation simplicity for catastrophic communication overhead, and on the swarm at these problem sizes, that trade was completely unfavorable.

n	t	T_S	T_O	S	E	pT_P
4,000	2	0.295	34.573	0.0169	0.0085	34.868
4,000	4	0.295	105.238	0.0112	0.0028	105.533
4,000	8	0.295	298.084	0.0079	0.001	298.379
4,000	16	0.295	1330.313	0.0035	0.0002	1330.608
8,000	2	1.319	135.637	0.0193	0.0096	136.955
8,000	4	1.319	427.785	0.0123	0.0031	429.104
8,000	8	1.319	1108.66	0.0095	0.0012	1109.979
8,000	16	1.319	4587.151	0.0046	0.0003	4588.47
16,000	2	6.987	626.907	0.022	0.011	633.894
16,000	4	6.987	1733.544	0.0161	0.004	1740.531
16,000	8	6.987	5056.723	0.011	0.0014	5063.711
16,000	16	6.987	16942.471	0.0066	0.0004	16949.458